Applied Reinforcement Learning

Tyler Bonnell

April 2026

1 Introductions and Definitions

1.1 Course Descriptions

Hi, I'm Tyler Bonnell an assistant professor in Computer Science. My research is largely in animal behaviour and data science.

Let's start by introducing what we will cover in this deep reinforcement learning class. Let's go over the structure of the course and how we'll take an applied approach. The course will consist of lectures to give us the background and theory we need to then run some algorithms in colab or your computer. By shifting between theory and application we will learn how to implement from scratch some modern DRL algorithms, and how to test them out in different environments.

The course itself will have 2 project assignments and 4 quizzes. The assignments are mean for you to practice your skills at setting up, implementing, and testing your algorithms. Gen AI is allowed, but you should be prepared to answer questions about your code. Failure to answer those questions sufficiently can result in a drastic grade drop. So just make sure you understand the code. The quizzes on the other hand are meant to simply help you retain and capture the higher-level concepts and details of DRL algorithms. We will not expect you to write code in the quiz, but you might have to read some. The quizzes will be in class, allowing us to learn about your understanding of reinforcement learning.

The syllabus is available on D2L and describes in more detail the course structure and marking. If you have any questions, feel free to reach out to me: tyler.bonnell@ucalgary.ca or via the class discord.

In terms of communication, we'll use D2L to hand out and submit assignments, and we will use discord to communicate during and outside class. I encourage you to use discord to discuss with other classmates, post what is working or not on your end, ask questions, and send me or the TA direct messages.

The following class notes are intended to give you more details, and they follow along with the class slides and tutorials. Much of these class notes are taken from the book by Sutton and Barto "Reinforcement learning: An Introduction." This book is available for free online (a link is provided in D2L).

1.2 What is Reinforcement Learning?

Before we start with the details, I want to give some context about RL. Most of us have had experience with supervised learning, likely in a machine learning or statistical context. With supervised learning we have a dataset that we fit a model on. That is, our supervised learning model extracts a pattern from the data that can take some inputs X and can then output a prediction y. We can then use the model to predict new data, test the generalizability of our model (i.e., how well the pattern it learnt can be used with other data), or maybe to infer something about the relationships that lead y to increase or decrease. Once we trust the model can make good predictions, and we understand how it does so, we might use the model in an applied context, predicting: whether a patient has ADHD, cancer, or diabetes, or maybe use it to predict what the weather will be tomorrow or next week. Under this supervised framework we extract patterns from data and use those to do some work. What happens, however, when the processes that generated the data changes? We might start to see that our predictions are becoming worse, and we'd have to update our model.

You might notice too that the process of extracting patterns from our data is not generally tied to how taking actions might impact outcomes. That is our model is made for prediction using the observed data, and not predictions for taking specific actions. This distinction can really matter. There is a whole field of causal analysis (or causal ML) who's goal it is to bridge this gap between prediction and causation. If we use our supervised learning model in context where the predictions are used to take actions, these actions can feedforward into the next batch of incoming data. In some cases, this can generate unintended biases in the data and the models being fit to this data.

Similarly, if we look at unsupervised learning, we find that the goal is to find structure in the observed data. Here structure could mean many things, and generally when choosing which algorithm to use when looking for structure we choose the one that most closely matches with our goals. If we are looking for mean representations of different classes of customers, we might choose a simple k-means clustering algorithm. What customer clusters are in the data? If we are trying to avoid objects with a self-driving car, we might want more flexibility in detecting object shapes, and choose something like HDBSCAN that lets a wide range of cluster shapes be delineated.

With unsupervised clustering, we are looking for explicit patterns in a data set already collected, or maybe in the case of the car, is being collected in real time. But we aren't making predictions or taking actions just yet.

We can look at RL as a machine learning approach that combines both supervised and unsupervised learning. Imagine you are a robot being thrown into an unknown world and you simply have to figure out what actions to take that lead to "good" outcomes. This is where RL can be very useful.

Though we will see that the use case does not always have to be a robot (e.g., robot exploring a factory floor) but can be a virtual bot trying to choose the next song for you (recommender systems), or to find the next buy/sell action

to take on the stock market (financial agent), or even to choose which source of energy to use (energy agent). In all these cases, you have an agent that might not be physical, though it does have to take actions, and lives in a world that is initially quite unknown. It doesn't know what music you like, but it can take actions (recommend songs) and it can record how successful the outcomes were (did you listen to it and maybe even "like" it?). Over time it might learn more about the world and what actions it can take to maximize successes, however success is defined for the agent.

Wherever you have this use case, an agent (physically embodied or not) that can take actions in an unknown world and must learn to act in a way that increases success... then reinforcement learning might be a good machine learning framework to choose!

Next, we will go over some of the terms we've been using more formally (what does success even mean?), as it will help us in building these reinforcement learning agents.

1.3 Agent-Environment interaction

We can start by framing the challenge of reinforcement learning as an agent learning what to do through interactions with an environment. The agent has three main aspects, sensation, action, and a goal, and is placed in an initially unknown environment. The agent and the environment then interact, with the agent choosing actions and sensing the response of the environment. The agent also receives a reward during interaction with the environment, this reward can be positive, negative, or even just zero.

If we consider discrete time steps $T = 1, 2, 3, \ldots, n$, then the agent can sense the current state of the environment $S_t \in \mathcal{S}$, and use that to choose an action $A_t \in \mathcal{A}(s)$. One time step afterwards, the agent receives a reward and finds itself in a new state. Information about state s can then influence what action a the agent takes, and these actions then update the environment and the state of the agent. These interactions then just keeps going, resulting in agent-environment interaction data that looks like the following:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, A_3, R_4, \ldots$$

Environmental states S can contain information about the state, such as position, or contain information about the state of the agent, such as how hungry it might be. Let's think about a simple grid environment where an agent might sit in one of the grid squares and its state could be defined as x, y in the gird. The agent could then transition to a new state S' in the grid by taking an action A_1. The actions could be random or they could follow some kind of policy π. This policy captures the agent's desire to take action a in state s. We can denote this using the following: $\pi(a \mid s)$. This policy might be terrible at first, i.e., the agent might simply move around randomly in our little grid. But if we provide it with a reward signal, something that says, the action you took in a particular state was a good one, then the agent might be able to improve its policy π.

The overall agent-environment interaction looks like the following:

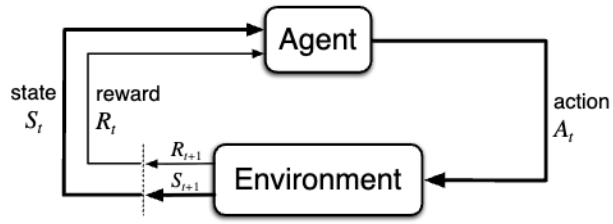

Figure 1: The agent-environment interaction in reinforcement learning (Sutton & Barto 1998).

One way to formalize agent-environment interactions is by using a Markov Decision Process (MDP), which defines the probabilities of transitioning from one state to another, how likely specific actions are in each state, how actions influence state transitions, and the rewards that the agent accumulates.

Let's break this down further:

- State (s): A state represents the set of values that describe the current context. This may include internal factors (e.g., hunger levels), external perceptions (e.g., current visual input), and even past experiences (e.g., previous visual images). A state can be composed of any combination of internal, external, or past/present variables.
- Transition Probabilities (p): The probability of transitioning from one state (s_1) to another state (s_2) is known as the transition probability. By interacting with the environment, the agent can learn these probabilities.
- Action (a): An action represents a set of possible behaviours that the agent can produce. These motor-based actions can be discrete (e.g., press a lever) or continuous (e.g., adjusting force applied to limbs). The availability of actions may also depend on the current state.
- Rewards (r): When an agent interacts with the environment, it may receive positive or negative signals. These signals can be thought of as internal (e.g., a release of dopamine when eating) or externally driven (e.g., receiving food as a result of an action).
- Policy (π): A policy is a strategy for selecting actions that maximize rewards. Ideally, the policy defines which action to take in each state to achieve the highest cumulative reward over time.

By breaking down an RL problem into these components, we can better understand the challenges an agent faces. From the agent's perspective, the

goal is to learn the optimal combination of states and actions that maximize cumulative rewards.

This description of an MDP assumes discrete states and actions. For example, an agent might choose the discrete action "eat", whereas in a continuous action space, the agent would instead adjust muscle forces (e.g., in the legs, arms, or torso) to produce eating behaviour dynamically. In later chapters we will explore how to handle continuous MDPs using Functional Approximation. However, the core principles of this simplified MDP framework remain relevant and serve as a valuable foundation for understanding reinforcement learning algorithms.

1.4 Rewards and Returns

We can see that in agent-environment interactions above the rewards play an important role. Providing a signal to the agent about what actions are good/bad. But what is a reward? We see in the MDP above that being in a state s and taking an action a we have some kind of impact on the environment, and as a result we get a reward r. In reinforcement learning, the goal of an agent is to gather as much rewards as it can. This doesn't always mean take actions that lead to the highest rewards immediately available, as there could be more rewards available if the agent takes other actions.

The accumulation of rewards act as a goal for our agents. We can state this as the reward hypothesis:

"That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward)."

We'll see that by setting up the rewards in different ways we allow our agents to prioritize different goals. This is a challenging part of applied reinforcement learning. For example, in health care settings we can use expert advice on how to setup rewards, such as negative rewards for patient discomfort or time-to-recovery, or positive rewards for range of motion achieved or time/stability on a prescribed medication. Though, there is always a large amount of care and monitoring to be done to make sure that in maximizing these rewards the agent is not taking actions that are harmful in other ways. I.e., We might give positive rewards for time on a prescribed medication as it avoids frequent switching of medications, but the agent might suggest medications that are more addictive and not necessarily better for patients.

If we want our agent to achieve something, we have to think carefully about how to give it rewards so that it achieves the thing we want. The idea here is that we set rewards so that the agent achieves the thing we want, while the agent figures out how to gain rewards.

Figure showing: Rewards as a way to learn behavior

By interacting with its environment the agent receives a reward signal: $R_{t+1}, R_{t+2}, R_{t+3}, \ldots$. The agent then tries to maximize the expected return of all these rewards where the return is some function of these rewards [sutton and barto 1989].

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T$$

Where T is the last time step of the agent-environment interaction. This way of measuring the return works quite well for episodic environments. That is, environments that have a clear termination state. Once the agent reaches this terminal state, the agent-environment interactions are over, and often the environment is reset back to the beginning state. You can think here of games like chess, where there is a terminal state (or states) like checkmate. Once checkmate is reached, the game is over.

But what about in cases where there is no terminal state and the environment goes on indefinitely? In those cases we can't just sum up the rewards, as we can end up with an infinitely growing expected return. Rather in these kinds of environments, we need to discount rewards that are farther into the future, and prioritize those that are near the present. If we sum up the discounted rewards we end up a finite estimate.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \tag{1}$$

$$= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2}$$

This parameter γ is called the discount rate and varies between 0 and 1. When set to a value of 0.01 the future rewards are only worth 0.01^{k-1} as much as rewards that are immediately available. This means the agent will prioritize actions that lead to immediate rewards and not rewards that might be gained in the future. If, on the other hand, the discount rate is set to 0.99, then future rewards are worth 0.99^{k-1}. This results in future rewards being worth almost as much as immediate rewards, and the agent prioritizing immediate and future rewards.

1.5 Policies and Value Functions

Now that we have data from agent-environment interactions and we know how to calculate returns we need to talk about how agents value certain states and even state-action pairs. The idea here is that the agent can learn which states, or state-actions pairs, are more valuable than others. A function that does this is called a value function. A value function estimates the future rewards expected in any state, in other words, it is a function that estimates the expected return of a state. Any future rewards of a state depends on what actions the agent takes in the future, so let's define another function, the policy function, that assigns probabilities to actions within a given state. So, if the agent is following policy π at time t the probability of action a would be $\pi(A_t|S_t)$. In reinforcement learning, we update this policy based on the agents experience so that the agent learns to takes actions that lead to the accumulation of more rewards.

The value function of a state s under policy π is then $v_\pi(S)$, and estimates the expected return of state s assuming the agent follows the policy π.

$$v_\pi(S) = E_\pi[G_t|S_t = s] \tag{3}$$

$$= E_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|S_t = s] \tag{4}$$

Here E_π is the expected value of a random variable given that the agent follows policy π. This function is called the state value function of policy π. Similarly, if we are defining the value of state-action pairs, where we want to track the value of an action a in state s following the policy π then we can define this function as:

$$q_\pi(s, a) = E_\pi[G_t|S_t = s, A_t = a] \tag{5}$$

$$= E_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|S_t = s, A_t = a] \tag{6}$$

We call q_π the action-value function for policy π.

We'll see that both the value-function v_π and the action-value function q_π can be estimated from experience. As the agent interacts with its environment the data it receives gives us a way to iteratively improve these functions, so that the agent gets better and better at estimating the value of states and actions within states. To solve a reinforcement learning problem generally means finding a policy that accumulates a lot of rewards. While in simple problems/environments it's possible to prove you've found the optimal policy, for most of the environments and applied applications we'll focus on in class we can only approximate the optimal policy, but not prove that we've found the true optimal policy.

Before moving on to how agents can learn to update its value-function and policy, let's complete the first and second tutorials where we will: create an environment, collect data on agent-environment interactions, and even put into practice a policy (even if it's currently a bad policy)!

For more in-depth review of this material see chapter 3 of Sutton and Barto.

1.6 Tutorial 1

In this tutorial, we will setup our first environment and see how we can collect data from an agent interacting with the environment. You should be able to open this tutorial notebook in colab, where most of the required python libraries are pre-installed, or on your own computers, where you might have to do some extra work installing the required libraries.

Learning outcomes:

- Setup an episodic environment

- Use a random policy to chose actions in states
- Collect agent-environment interaction data

1.7 Tutorial 2

In this tutorial, we will build custom policies as neural networks. We'll see that depending on the types of actions that are possible in your environment we have to setup the policy slightly differently. This largely breaks down to if the actions taken are continuous (e.g., how much force to apply, and at what angle?), or discrete (e.g., move left or right, press button a or b?).

Learning outcomes:

- Build a policy from a neural network
- Build a policy for discrete and continuous action spaces
- Use the policy to gather agent-environment interaction data

2 Learning Better Policies and Value Functions

2.1 Bellman equation

The basis of many reinforcement learning algorithms is the Bellman equation. This equation tells us how exactly agents can learn from their interactions with their environments, and how the value of the current state is a function of the value of a future state. This recursive nature is how the agent can make predictions about the value of a state, take action and see how far off that predicted value was, and then how to update it's value function to make better predictions in the future.

We can get the Bellman equation from our value function equations above.

$$
\begin{aligned}
v_\pi(s) &= E_\pi[G_t|S_t = s] && (7)\\
&= E_\pi[R_t + \gamma G_{t+1}|S_t = s] && (8)\\
&= R_t + \gamma E_\pi[G_{t+1}|S_{t+1} = s'] && (9)\\
&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a)[r + \gamma E_\pi[G_{t+1}|S_{t+1} = s']] && (10)\\
&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a)[r + \gamma v_\pi(s')], \quad \text{for all } s \in \mathcal{S} && (11)\\
& && (12)
\end{aligned}
$$

This might look like a lot, but really what the main take-away here is that we started with a familiar definition of our value function, i.e., the value of a state is the expected returns from that state assuming the agent follows policy

π. But after some algebra we end up with a definition of our value function that is based on the value function of the next state! That is $v_\pi(s)$ is related in some way to $v_\pi(s')$. We can use this fact to help update our value function, we'll see how below, but first let's walk through each step above.

The first equation above is just our definition of the value function. In the second line we note that the current return G_t can be rewritten as $R_t + \gamma G_{t+1}$. That is, the current return is just the observed reward received plus the discounted rewards of all future rewards. Then as the observed reward is just a random variable and the discount rate is applied to a sum, we can move the expectation inside to just have the expectation of the future returns. We can then re-write the observed rewards R_t as a multiplication of two terms: the probability of each action, and the product of the probability of reaching a particular next state and receiving a particular reward. The final line is just recognizing that the expectation of the returns starting from the next state is just like the value function definition in the first line, just applied to the next time point. So we replace the expectation with the value function just at the next state.

2.2 TD-Learning

2.2.1 Building our intuition

Now that we understand the components of a reinforcement learning problem, let's explore how an agent might learn which states are valuable and lead to rewards. The first step is to create a function that will give us the value of a particular state, that's just our value function $V_\pi(S)$. This is what we want our agent to learn. But how does an agent update its estimate of a state's value based on experience?

We need a method that allows the agent to adjust its current estimate of the state's value, $V_\pi(s_t)$, after interacting with the environment. The updated estimate should ideally move closer to the true expected return from being in that state. We can express this as follows:

$$V(s_t) = V(s_t) + \text{some update value} \tag{13}$$

Or more simply, we can express this as a change i.e., δ:

$$\delta = \text{some update value} \tag{14}$$

There are many possible ways to update the expected value of a state. A theoretially optimal way of updating the value of a state is to use the Bellman equation, and to update a state's value by comparing what the agent expected to happen with what actually happened in the next state. Specifically, the agent looks at the reward it just received, adds the expected value of the next state, and compares that total to the value it previously assigned to the current state.

The difference between these two quantities is used to update the current state's value.

At first, this may seem strange or even counterintuitive, why use the value of the next state to update the current state? The reason is that by incorporating information from future states, we allow the agent to learn sequences of behaviours that reliably lead to rewarding outcomes. To build our intuition about this process, let's examine a simple example.

Let us use as our example, a gerbil agent that moves from trash can to trash can looking for food. When in a trash can we will consider that as a different state for our gerbil (see Fig. 2), and each day the gerbil agent wakes up hungry. We will also force the gerbil agent to start in the leftmost trash can S_0 each day and force it to move one can to the right until it reaches the last trash can S_3. When the gerbil reaches the last trash can on the right (i.e., end state), the training episode ends, and we'll move the gerbil back to the starting state S_0 to begin another episode on the next day. Finally, we'll place some food in the last trash can, where the agent reliability finds some old pizza, and can reduce its hunger. The last trash can is then a source of potential reward for our agent $R_3 = 1$.

We can make an MDP of this gerbil environment as follows:

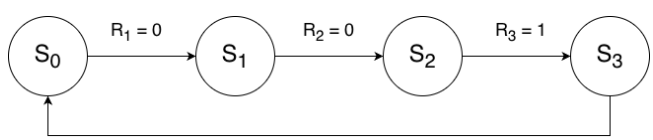

Figure 2: MDP representation of the gerbil environment.

Let's examine how the gerbil agent gradually learns the value of each state (i.e., trash can) by using the total value of the next state to update its estimated value for the current state. To do this let's look at three training episodes one at a time.

Episode 1: The gerbil agent starts in S_0 and goes from trash can to trashcan and finally finds a reward when arriving in S_3. The gerbil updates the expected value of moving from S_2 to S_3 positively. The gerbil is developing an expectation that the 3rd trash can leads to a reward!

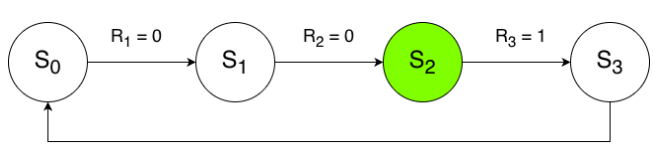

Figure 3: MDP representation of the gerbil environment: $V(S_2)$ increased.

Episode 2: The gerbil starts again in state S_0 and goes from trash can to trash can and when it reaches S_1 it increases that state's value, as S_2 now has

some expected value. The estimated values are getting transferred to earlier states. Once in the S_2, the move to S_3 again results in pizza, a reduction in hunger, and therefore a reward. The gerbil then updates S_2 expected value to be higher again!

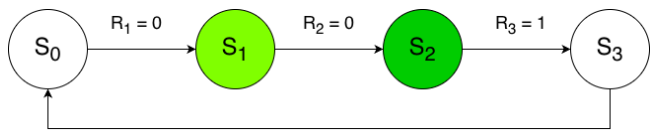

Figure 4: MDP representation of the gerbil environment: $V(S_1), V(S_2)$ increased.

Episode 3: The gerbil starts over again from S_0 and now it increases the state value of S_0 as going to S_1 has some expected value. The other trash cans are again updated positively, as they all reliably lead to pizza!

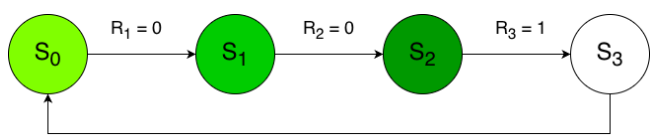

Figure 5: MDP representation of the gerbil environment: $V(S_0), V(S_1), V(S_2)$ increased.

The key takeaway is this: if an agent finds a rewarding state, and other states reliably lead to that rewarding state, the estimated value of states effectively spreads to those preceding states. Using this approach, the agent can learn behavioural sequences that lead to a rewarding outcome. In our simplified example, we forced the agent through states in a left-to-right sequence, so it is not surprising that the agent learns that each state reliably leads to the next one, ultimately reaching the rewarding state at the end.

But how exactly can the agent calculate the value of states that reliably lead to rewarding outcomes?

2.2.2 Temporal Difference Learning

Let us start off by simplifying our gerbil example above even more to just two states and introduce equations that will let us update the values of states.

Let's take a moment to go over each part of this new MDP diagram (Fig. 6). First, there are two states: S_a and S_b. For simplicity, let's assume that when the agent is in S_a, it always transitions to S_b with probability $p_{a,b} = 1.0$. Likewise, when the agent is in S_b, it always transitions back to S_a with probability $p_{b,a} = 1.0$. This is just like the previous example. We therefore have fixed the transition probabilities, meaning that the agent cannot choose to stay in a particular state, it must alternate between the two states. We do this just

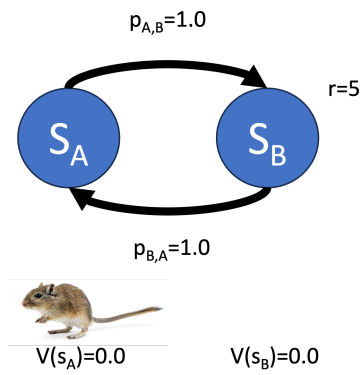

Figure 6: MDP representation of the gerbil environment: $V(S_0), V(S_1), V(S_2)$ increased.

to make the example as simple as possible, i.e., the agent at the moment cannot really choose any other actions. Finally, when the agent transitions to S_b, it eats and receives a reward of +5 for reducing its hunger.

Using this simple example, let's walk through how an agent learns that state S_a leads to rewards, and what value it should assign to S_a. Let's start off by setting the expected value of each state to 0, $V(S_a) = 0$ and $V(S_b) = 0$, and placing the gerbil in S_a (6). To then update the value of the current state S_a we could initially focus on using the observed reward received at the next state R_{t+1}. We can write that as:

$$\text{change state value} = \text{observed rewards}_{\text{next state}} \tag{15}$$

$$\delta = R_{t+1} \tag{16}$$

However, if we update the state value this way, the expected reward for S_a will just keep increasing each time the gerbil reaches S_b. For example, if the gerbil receives a reward of +5 each time it reaches S_b, then the value update for S_a would simply accumulate. This does not make sense, as the actual reward for transitioning from S_a to S_b is always 5, not an ever-growing sum. Instead, we should update the state value based on the difference between the observed reward at the next state and the current estimated value of S_a. This can be expressed as:

$$\text{change state value} = \text{observed rewards}_{\text{next state}} - \text{expected value}_{\text{current state}} \tag{17}$$

$$\delta = R_{t+1} - V(S_t) \tag{18}$$

Let's apply this method to our gerbil example. If the gerbil starts with

$V(S_a) = 0$ and $V(S_a) = 0$, when the gerbil moves from s_A to s_B and receives a reward of 5 we can update the value of S_a as follows:

$$\delta = R_{t+1} - V(S_a) \tag{19}$$
$$\delta = 5 - 0 \tag{20}$$
$$\delta = 5 \tag{21}$$

So, now the gerbil will change its estimated value of S_a by 5:

$$V(S_{a,t+1}) \leftarrow V(S_{a,t}) + \delta \tag{22}$$
$$V(S_{a,t+1}) \leftarrow 0 + 5 \tag{23}$$
$$V(S_{a,t+1}) \leftarrow 5 \tag{24}$$

This method ensures that $V(S_a)$ converges to the actual expected reward, rather than increasing indefinitely. Let's make sure this is the case, and let's move our gerbil again! Now that the gerbil is in S_b, let's move her back to S_a. When the gerbil goes from S_b to S_a it will update the value of S_b by:

$$\delta = R_{t+1} - V(S_b) \tag{25}$$
$$\delta = 0 - 0 \tag{26}$$
$$\delta = 0 \tag{27}$$

So now our gerbil still estimates that the value of S_b is zero:

$$V(S_{b,t+1}) = V(S_{b,t}) + \delta \tag{28}$$
$$V(S_{b,t+1}) = 0 + 0 \tag{29}$$
$$V(S_{b,t+1}) = 0 \tag{30}$$

Let's move our gerbil one more time and get it to go from S_a to S_b again. It will update the value of S_a by:

$$\delta = R_{t+1} - V(S_a) \tag{31}$$
$$\delta = 5 - 5 \tag{32}$$
$$\delta = 0 \tag{33}$$

So, because our gerbil estimates that the value of S_a is 5 and it receives a reward of five by going into S_b, it sees no difference between what it expected and what it observed. So, in this case the gerbil updates its value of S_a by 0.

$$V(S_{a,t+1}) = V(s_(A, t)) + \delta \tag{34}$$
$$V(S_{a,t+1}) = 5 + 0 \tag{35}$$
$$V(S_{a,t+1}) = 5 \tag{36}$$

So, the values stabilize and remain $V(S_a) = 5$ and $V(S_b) = 0$.

We can see that the gerbil learns that state S_a leads to rewards, while state S_b does not. However, we might consider allowing the agent to look further into the future. By allowing the agent to look ahead further, it might recognizing that state S_b can lead to state S_a, which then leads back to S_b, ultimately bringing a reward!

Being able to anticipate rewards over multiple steps can be especially useful in more complex environments, where behavioural sequences leading to a reward may be longer. The key to achieving this is to incorporate the expected value of future states into the total expected reward using a discount factor (γ). Remember that when the discount factor is one ($\gamma = 1$), the agent values future rewards just as much as immediate rewards, meaning it considers long-term outcomes. But when the discount factor is zero ($\gamma = 0$), the agent only values immediate rewards, just as we saw in the gerbil example above.

We will explore how to tune this parameter, but for now, let's see how our agent performs with a discount factor of $\gamma = 0.5$. To incorporate discounted rewards when updating the value of a state we will simply add the expected rewards of the next state times the discount factor. Let's take a look at how this works below:

$$\text{Change in state value} = \text{Observed reward}_{\text{next state}} \tag{37}$$
$$+ \text{discount} \times \text{expected value}_{\text{next step}} \tag{38}$$
$$- \text{expected value}_{\text{current step}} \tag{39}$$

$$\delta = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \tag{40}$$

To see how this works let's start over with a new gerbil in the same MDP setup. This new gerbil will start with no expectation of rewards in either state: i.e., $V(S_a) = 0$ and $V(S_b) = 0$, and the gerbil will start in S_a and we will force it to move to S_b. When the gerbil takes its first step going from S_a to S_b it will update the value of S_a using the following equation:

$$\delta = R_{t+1} + \gamma * V(S_{t+1}) - V(S_t) \tag{41}$$
$$\delta = R_{t+1} + 0.5 * V(s_b) - V(s_A) \tag{42}$$
$$\delta = 5 + 0.5 * 0 - 0 \tag{43}$$
$$\delta = 5 \tag{44}$$

So, in this case this new gerbil updates the value of state S_a just like the first gerbil:

$$V(S_a) \leftarrow V(S_a) + \delta \tag{45}$$
$$V(S_a) \leftarrow 0 + 5 \tag{46}$$
$$V(S_a) \leftarrow 5 \tag{47}$$

Let's move the gerbil again. When the gerbil goes from S_b to S_a it will update $V(S_b)$ by:

$$\delta = R_{t+1} + \gamma * V(S_{t+1}) - V(S_t) \tag{48}$$
$$\delta = R_{t+1} + 0.5 * V(S_a) - V(S_b) \tag{49}$$
$$\delta = 0 + 0.5 * 5 - 0 \tag{50}$$
$$\delta = 2.5 \tag{51}$$

So now the value of state S_b is updated to 2.5 as the gerbil is now taking into account the fact that S_a leads to S_b:

$$V(S_b) \leftarrow V(S_b) + \delta \tag{52}$$
$$V(S_b) \leftarrow 0 + 2.5 \tag{53}$$
$$V(S_b) \leftarrow 2.5 \tag{54}$$

Let's move the gerbil one more time, and move it back to S_b. This time the gerbil will update $V(S_a)$ using:

$$\delta = R_{t+1} + 0.5 * V(S_b) - V(S_a) \tag{55}$$
$$\delta = 5 + 0.5 * 2.5 - 5 \tag{56}$$
$$\delta = 1.25 \tag{57}$$
$$\tag{58}$$

So now the gerbil estimates the value of state A to be increased by 1.25:

$$V(S_a) \leftarrow V(S_a) + \delta \tag{59}$$
$$V(S_a) \leftarrow 5 + 1.25 \tag{60}$$
$$V(S_a) \leftarrow 6.25 \tag{61}$$

If we keep going for a few more iterations the values start to stabilize, and remain $V(S_a) = 8.0$ and $V(S_b) = 3$. So now our gerbil learns that state S_b does have some future reward. We also see that the value of state S_a is above 5! That means by having our gerbil look to future values (discounted by the discount factor), some of the value from visiting state S_b in future states is being incorporated into its expectation of total rewards. Resulting in the gerbil valuing state S_a above what it might get in rewards from moving to state S_b in

one step. Clever gerbil!

Next, we need to introduce one final parameter called the learning rate (α). This parameter determines how quickly the agent updates its estimated rewards. The learning rate can be thought of as how fast the agent adapts to reward signals. If α is too high, the agent updates its reward predictions drastically with every new observation, leading to erratic behaviour. If α is too low, the agent learns very slowly, making it difficult to adapt efficiently. To complete our value-based update equation, we can add the learning rate parameter as follows:

$$V(S_a) \leftarrow V(S_a) + \alpha\delta \tag{62}$$

You can try out different levels of learning rates (e.g., $0 < \alpha \leq 1$) with the examples above if you want to see how this changes the results.

What we have introduced here is called the temporal difference learning, or TD-learning. This method updates value estimates based on the difference in reward predictions between time steps—a process known as "bootstrapping." The term bootstrapping refers to how the agent "pulls itself up by its bootstraps"—it uses its own past experiences to refine its estimates of state values. Given enough time, an agent exploring its environment can gradually learn accurate state values.

From a biological point of view, TD-learning closely aligns with how dopamine release functions in animals. Animals show dopamine release in anticipation of a reward, and changes in dopamine firing patterns occur when actual rewards deviate from expected rewards. This deviation between expected and actual rewards is called a reward prediction error – or TD error – and is central to how the TD-learning algorithm works. If you want to know more about the similarities and differences between these type of algorithms and learning in animals see Ch: 14-15 in the Sutton and Barto book.

But for now, let's do a tutorial on how to calculate the TD-target and update the value of states and state-action pairs for a reinforcement learning task.

3 TD Learning in practice

Now that we know how TD learning works, let's use this approach with some agents in different environments. We are going to do this in two ways: on-policy and off-policy. Here on-policy refers to the cases where an agent is following policy π and updates this policy with data collected using this policy. In the case of off-policy, the agent can have a policy π but might act differently than that policy to collect that data. Essentially, anytime data was collected with a different policy, i.e., a random policy, or even an older policy, and those data

are used to update the current policy this is off-policy learning. Whereas with on-policy the data are collected with the current policy.

3.1 SARSA (on-policy)

Let's start off with the on-policy case here and take a look at the SARSA algorithm. Remember, that when the agent is interacting with the environment we are generating data that looks like:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, A_3, R_4, \ldots$$

We can use these sequences and the TD learning approach to learn the value of state-action pairs so that the agent can take better and better actions in particular states. Using a TD-learning approach this would look like the following:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \tag{63}$$

After each agent environment interaction we can then gather the sequence: $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$ and then use these data to update $Q(S_t, A_t)$ after each step. When we reach the end of the agent-environment interactions, this could be the agent getting out of maze, we call this the terminal state, and the value of $Q(S_t, A_t)$ is zero.

Using this approach, as the agent interacts with the environment over time it will update these q-values (i.e., $Q(S_t, A_t)$) and get better at predicting the right value for each state-action pair. We could then let the agent choose an action A_t based on the value of the actions in that state, so that actions that are estimated to be of a higher value are more likely to be picked. Generally, a softmax function is used to probabilistically choose actions:

$$P(a|s) = \frac{e^{Q(s,a)}}{\sum_i e^{Q(s,a_i)}} \tag{64}$$

This means that as the agent starts to learn that some state-action pairs lead to better rewards, it can start to chain together sequences of state-action pairs to accumulate higher and higher returns. Using a softmax approach the agent can find high value sequence of state-action pairs, and still sometimes explore currently low value state-action pairs in case they lead to more rewards.

There are, however, other ways we can help the agent balance this trade-off between exploiting what it knows and exploring what might be more rewarding. We could use instead what is called an $\epsilon-$greedy approach. With this approach we define a value ϵ that represents the probability that the agent will either choose the highest value action action based on the current q-values, or choose an action at random. By choosing at random the agent will be able to start to explore alternative state-action pairs that might not be currently highly valued

but might actually lead to higher rewards. Using this simple approach can sometimes be quite effective, as initial the agent's q-value estimates tend to be very noisy so the extra computation of softmax might not be worth it. Similarly, if we let the value of ϵ to slowly decrease, as the agent gets better estimates of the q-values it can take more advantage of these high rewarding state-action pairs.

$$a = \begin{cases} \text{random} & \text{with probability } \epsilon \\ \arg\max_a Q(s,a) & \text{with probability } 1-\epsilon \end{cases} [10pt] \tag{65}$$

3.2 Tutorial 3

In this tutorial, we'll start collecting some agent-environment interaction data and start to use this data to allow our agents to improve their action-value functions. As they interact with their environment they can start to choose actions in states that lead to the accumulation of more and more rewards. We will see that there are pre-made functions that can help us to calculate the necessary updates, but we will also do this manually so we will know how these updates work in practice.

Learning outcomes:

- Calculate q-values of actions in states
- Use TD-learning to update our action-value function so that it makes better predictions and chooses better actions
- Use the action-value function to gather agent-environment interaction data

3.3 Deep Q-networks (off-policy)

Let us now look at the off-policy case. We will see that deep q-networks (DQN) is a very similar algorithm compared to SARSA. The main difference is that DQN is off-policy and SARSA is on-policy. While SARSA samples agent-environment interactions and updates it's action values each time step, DQN does something slightly differently.

Replay buffers: DQN samples agent-environment interactions, and then places them in a replay buffer. This replay buffer is usually given a fixed size, and when it fills up, new agent-environment interactions replace older ones. The replay buffer is therefore filled with recent agent-environment interactions. When learning from this replay buffer we can sample randomly, using state, action, reward, next state, next action sequences to update the q-values.

Max action: When learning DQN uses almost the exact same TD approach as SARSA with a little more optimistic outlook. Here is the DQN version of

the TD learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \tag{66}$$

The only difference between this equation and the SARSA one is that we are assuming that in the next state the agent will take the best action, i.e., $\max_a Q(S_{t+1}, a)$ and not the actual action that was taken.

Off-policy: DQN is then an off-policy algorithm as it learns by samples from a replay buffer where the sequences of agent-environment interactions might have been generated by an older policy. Similarly, it learns by assuming the action taken in the next time step is the best one and not the actual one taken using the current policy.

On-policy vs. Off-policy learning: does the algorithm learn from data collected by the current policy (on-policy), or from stored experience (off-policy)?

Stability: DQN has been a very useful RL algorithm, however, it suffers from stability issues. It's been found that if an RL algorithm is off-policy, using neural networks (or any kind of functional approximation), and uses TD like methods (that learn by bootstrapping as they go), then stability problems tend to arise. This is known as the deadly triad! If you'd like to read more check out chapter 6 of Sutton and Barto.

There has been some good work on how the DQN algorithm is implemented to make it more stable. These implementation methods are a useful way to think about how RL algorithms learn and their pathologies. Let's take a look at two main ways we can help our agents learn with DQN.

3.4 Implementation of DQN

Double network: There is a problem with always selecting the best action. If there is noise in the estimated values of state-action pairs, which there most assuredly is, especially at the start of learning. Then that noise will push some value estimates higher or lower than they should be. Taking the maximum of those values results in a positive bias, with the maximum being more often than not being higher than it should. This can cause DQN agents to consistently over estimate the value of state-action pairs.

To solve this issue, it has been found that creating two networks and using one to decide which action is best, and the second to estimate the value of that state-action pair. By decoupling this process, the network that chooses which action is best in a state is likely to still overestimate the value of that state-action pair, but the second network will have its own noise, uncorrelated with the first network, and as a result, will likely have a more sober estimate of that state-action pair.

Let us separate the action selection from the valuation a little more explicitly in the TD learning equation, and specify where the target network copy will be used. The updated TD learning looks like the following:

$$a^* = \arg\max_a Q_{net1}(S_{t+1}, a) \tag{67}$$

$$Q_{net1}(S_t, A_t) \leftarrow Q_{net1}(S_t, A_t) + \alpha[R_{t+1} + \gamma Q_{net2}(S_{t+1}, a^*) - Q_{net1}(S_t, A_t)] \tag{68}$$

So we can see we are using this second q-network to calculate the TD-target, i.e., estimating what the value of the best action in the next state is likely to be: $Q_{net2}(S_{t+1}, a^*)$. While first q-network is used everywhere else to interact with the environment in the current moment, selecting the best current action and estimating the value of the current state-action pair $Q_{net1}(S_t, A_t$.

Most often this second network is a copy of the first q-network, that is then slowly updated to match the first q-network, e.g., updated every so often rather than every step. With this DQN setup, the second network helps with the overestimation process, while implementing a slow updating of this second network results in more stable TD-targets.

To read more in-depth about the topics covered in this section read chapter 6 from Sutton and Barto.

3.5 Tutorial 4

Now that we know how to calculate q-values and to improve our action-value functions, let's put this into practice and solve an environment. Remember, to solve a task in this applied RL we would like to see our agents accumulating a large amount of rewards. We'll see that what we have learned so far allows us to build deep q networks (DQN), and we will see that there are some ways to improve this algorithm in practice (double DQN). We will also learn functions to facilitate the collection and use of agent-interaction data when learning a better and better action-value function.

Learning outcomes:

- Setup DQN with a data collector to collect and process agent-environment interaction
- Implement double DQN
- Do some hyper-parameter sensitivity testing

4 Actor-Critic Methods

5 Model Based Reinforcement Learning

6 Data sets and environments

6.1 Gymnasium

6.2 Pettingzoo

6.3 pufferAI

Might be too much for class (requires cuda etc...) https://puffer.ai/index.html

6.4 Stock Market

Might not be a good idea...

6.5 Energy Grid

Paper describing challenge: https://arxiv.org/pdf/2003.07339
Environment (notebooks with code available): https://github.com/Grid2op/grid2op

6.6 Drone

6.7 Recommender Systems